LANCHONETE EXPRESS

Sistema de Pedidos Online

Documento de Planejamento Completo

Trabalho Final — Curso de Programador Web

1. Estrutura Geral do Sistema

O projeto é dividido em três camadas principais que se comunicam entre si:

🖥️ FRONTEND	⚙️ BACKEND	🗄️ BANCO DE DADOS
HTML + CSS + JS	Node.js + Express	MySQL
Hospedado no Vercel	Hospedado no Render	Hospedado no Railway
Consome a API REST	Expõe rotas REST	Armazena todos os dados
Site do cliente + Painel admin	Regras de negócio	Usuários, produtos, pedidos

Como os três se comunicam:

- O Frontend envia requisições HTTP (fetch) para o Backend
- O Backend processa a lógica e consulta o MySQL
- O MySQL retorna os dados, o Backend envia JSON para o Frontend
- O Frontend renderiza os dados na tela para o usuário

1.1 Diagrama de Comunicação

```
USUÁRIO → Navegador (HTML/CSS/JS) → fetch() → API Express (Node.js) → MySQL
←──────────────── resposta JSON
──────────────────────────────────────────────────────────────────────────
```

2. Arquitetura - MVC Simplificado

Padrão MVC (Model - View - Controller), simples, organizado e muito usado no mercado. Veja como funciona:

Camada	O que é	Exemplo no projeto
Model	Acessa o banco de dados	productModel.js — faz SELECT no MySQL
View	O HTML/CSS que o usuário vê	index.html, admin.html
Controller	Recebe requisição, chama o Model, envia resposta	productController.js — conecta rota com model
Routes	Define os endpoints da API	productRoutes.js — GET /api/products

> Dica Didática: Pense assim: a VIEW é o cardápio que o cliente vê. O CONTROLLER é o garçom que anota o pedido. O MODEL é a cozinha que prepara e entrega o prato. As ROUTES são a mesa onde o pedido chega.

Estrutura de Pastas do Projeto

O projeto terá um único repositório no GitHub com dois diretórios principais:

```
lanchonete-express/
├── frontend/
│   ├── index.html          # Página inicial (produtos)
│   ├── login.html          # Tela de login
│   ├── cadastro.html       # Tela de cadastro
│   ├── carrinho.html       # Carrinho de compras
│   ├── pedido.html         # Finalizar pedido + QR Code
│   ├── admin/
│   │   ├── index.html      # Painel administrativo
│   │   ├── produtos.html   # Gerenciar produtos
│   │   ├── pedidos.html    # Ver pedidos
│   │   └── promocoes.html  # Criar promoções
│   ├── css/
│   │   ├── style.css       # Estilos globais
│   │   └── admin.css       # Estilos do painel
│   ├── js/
│   │   ├── api.js          # Funções para chamar o backend
│   │   ├── carrinho.js     # Lógica do carrinho
│   │   ├── cep.js          # Integração API de CEP
│   │   ├── auth.js         # Login e sessão do usuário
│   │   └── admin.js        # Scripts do painel admin
│   └── images/             # Imagens dos produtos
│
├── backend/
│   ├── server.js           # Ponto de entrada do servidor
│   ├── .env                # Variáveis de ambiente (senhas)
│   ├── .env.example        # Exemplo do .env (vai pro GitHub)
│   ├── config/
│   │   └── db.js           # Conexão com MySQL
│   ├── models/
│   │   ├── userModel.js    # Consultas de usuário
│   │   ├── productModel.js # Consultas de produto
│   │   └── orderModel.js   # Consultas de pedido
│   ├── controllers/
│   │   ├── userController.js
│   │   ├── productController.js
│   │   └── orderController.js
│   ├── routes/
│   │   ├── userRoutes.js
│   │   ├── productRoutes.js
│   │   └── orderRoutes.js
│   ├── middleware/
│   │   └── auth.js         # Proteção de rotas admin
│   └── package.json
│
├── database/
│   └── schema.sql          # Script de criação do banco
│
└── README.md               # Documentação do repositório
```

4. Estrutura do Banco de Dados

O banco de dados MySQL terá 5 tabelas principais. Veja cada uma com suas colunas e o motivo de cada campo:

4.1 Tabela: usuarios

Coluna	Tipo	Chave	Descrição
id	INT AUTO_INCREMENT	PK	Identificador único do usuário
nome	VARCHAR(100)		Nome completo
email	VARCHAR(150) UNIQUE		E-mail para login (único)
senha_hash	VARCHAR(255)		Senha criptografada com bcrypt
telefone	VARCHAR(20)		WhatsApp para contato
cep	VARCHAR(9)		CEP do endereço
rua	VARCHAR(200)		Logradouro
numero	VARCHAR(10)		Número da residência
bairro	VARCHAR(100)		Bairro
cidade	VARCHAR(100)		Cidade
estado	CHAR(2)		UF ex: BA, SP
criado_em	TIMESTAMP DEFAULT NOW()		Data de cadastro

4.2 Tabela: produtos

Coluna	Tipo	Chave	Descrição
id	INT AUTO_INCREMENT	PK	Identificador único
nome	VARCHAR(150)		Nome do produto ex: X-Burguer
descricao	TEXT		Ingredientes e detalhes
preco	DECIMAL(10,2)		Preço ex: 18.90
preco_promocional	DECIMAL(10,2) NULL		Preço com desconto (se houver)
categoria	VARCHAR(50)		Lanches, Bebidas, Sobremesas...
imagem_url	VARCHAR(255)		URL da imagem do produto
disponivel	TINYINT(1) DEFAULT 1		1=disponível, 0=indisponível
criado_em	TIMESTAMP DEFAULT NOW()		Data de cadastro do produto

4.3 Tabela: pedidos

Coluna	Tipo	Chave	Descrição
id	INT AUTO_INCREMENT	PK	Número do pedido
usuario_id	INT	FK	Referência ao usuário
total	DECIMAL(10,2)		Valor total do pedido
status	ENUM(...)		'pendente', 'confirmado', 'entregue', 'cancelado'
endereco_entrega	TEXT		Endereço formatado completo
observacao	TEXT NULL		Observações do cliente
criado_em	TIMESTAMP DEFAULT NOW()		Data e hora do pedido

4.4 Tabela: itens_pedido

Esta tabela conecta pedidos e produtos — um pedido pode ter vários produtos.

Coluna	Tipo	Chave	Descrição
id	INT AUTO_INCREMENT	PK	Identificador único do item
pedido_id	INT	FK	Referência ao pedido
produto_id	INT	FK	Referência ao produto
quantidade	INT		Quantos foram pedidos
preco_unitario	DECIMAL(10,2)		Preço na hora do pedido (histórico)

4.5 Tabela: administradores

Coluna	Tipo	Chave	Descrição
id	INT AUTO_INCREMENT	PK	Identificador único
nome	VARCHAR(100)		Nome do administrador
email	VARCHAR(150) UNIQUE		E-mail para login
senha_hash	VARCHAR(255)		Senha criptografada
criado_em	TIMESTAMP DEFAULT NOW()		Data de criação do admin

4.6 Relacionamentos

usuarios (1) ——— (N) pedidospedidos (1) ——— (N) itens_pedidoprodutos (1) ——— (N) itens_pedidoUm usuário pode ter muitos pedidos.Um pedido pode ter muitos itens.Um produto pode aparecer em muitos itens.

5. Fluxo Completo do Usuário no Site

Passo a passo de tudo que o cliente faz, desde o acesso até receber a confirmação:

#	Etapa	O que acontece
1	Acessa o site	Vê a página inicial com os produtos cadastrados, preços e fotos
2	Cadastro	Preenche nome, e-mail, senha, telefone e endereço (CEP preenche automático via API)
3	Login	Informa e-mail e senha, recebe um token JWT armazenado no localStorage
4	Ver cardápio	Produtos listados por categoria com nome, foto, descrição e preço
5	Adicionar ao carrinho	Clica em 'Adicionar' — produto vai para o carrinho salvo no localStorage
6	Ver carrinho	Vê todos os itens, quantidades, subtotais e total geral
7	Finalizar pedido	Confirma endereço de entrega e observações
8	Pedido salvo	Backend salva o pedido no MySQL com status 'pendente'
9	QR Code gerado	Sistema gera QR Code com link do WhatsApp já com a mensagem do pedido
10	Envio pelo WhatsApp	Cliente escaneia o QR Code e envia a mensagem para a lanchonete

6. Fluxo Completo do Administrador

#	Ação	Como funciona
1	Login admin	Acessa /admin — autêntica com e-mail e senha de administrador
2	Dashboard	Ver resumo: total de pedidos do dia, faturamento, produtos mais vendidos
3	Adicionar produto	Preenche formulário com nome, preço, categoria, imagem e salva no banco
4	Editar produto	Altera qualquer campo do produto e salva a atualização
5	Remover produto	Deleta o produto (ou marca como indisponível para preservar histórico)
6	Criar promoção	Define um preco_promocional para um produto já existente
7	Ver pedidos	Lista todos os pedidos com cliente, itens, valor e status
8	Atualizar status	Muda o status do pedido: pendente → confirmado → entregue
9	Fluxo de vendas	Gráfico ou tabela mostrando pedidos por dia/semana

Segurança: O painel administrativo deve exigir autenticação. Todas as rotas /api/admin/* devem verificar se o token JWT pertence a um administrador. Sem isso, qualquer usuário poderia acessar o painel.

7. Rotas da API do Backend

Todas as rotas começam com /api. O frontend fará chamadas fetch() para estas rotas:

7.1 Rotas de Usuários — /api/users

Método	Rota	Auth?	Descrição
POST	/api/users/register	Não	Cadastrar novo usuário
POST	/api/users/login	Não	Login — retorna token JWT
GET	/api/users/profile	Usuário	Buscar dados do perfil logado
PUT	/api/users/profile	Usuário	Atualizar dados do perfil

7.2 Rotas de Produtos — /api/products

Método	Rota	Auth?	Descrição
GET	/api/products	Não	Listar todos os produtos disponíveis
GET	/api/products/:id	Não	Buscar um produto específico
POST	/api/products	Admin	Criar novo produto
PUT	/api/products/:id	Admin	Editar produto existente
DELETE	/api/products/:id	Admin	Remover produto

7.3 Rotas de Pedidos — /api/orders

Método	Rota	Auth?	Descrição
POST	/api/orders	Usuário	Criar novo pedido
GET	/api/orders/my	Usuário	Listar pedidos do usuário logado
GET	/api/orders	Admin	Listar todos os pedidos
GET	/api/orders/:id	Admin	Detalhes de um pedido específico
PUT	/api/orders/:id/status	Admin	Atualizar status do pedido

7.4 Rotas do Admin — /api/admin

Método	Rota	Auth?	Descrição
POST	/api/admin/login	Não	Login do administrador
GET	/api/admin/dashboard	Admin	Dados do dashboard (totais, gráficos)
GET	/api/admin/sales	Admin	Fluxo de vendas por período

8. Integração com a API de CEP

Utilizaremos a API ViaCEP — gratuita, sem necessidade de cadastro ou chave de API.

URL: https://viacep.com.br/ws/{CEP}/json/Exemplo: https://viacep.com.br/ws/01310100/json/

Como implementar no frontend (arquivo cep.js):

```
// Arquivo: frontend/js/cep.js
// Chamado quando o usuário sai do campo CEP (evento 'blur')
async function buscarCep(cep) {
  cep = cep.replace(/[^0-9]/g, '');  // Remove traços e espaços
  if (cep.length !== 8) return;       // CEP deve ter 8 dígitos

  try {
    const response = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
    const data = await response.json();

    if (data.erro) {
      alert('CEP não encontrado!');
      return;
    }

    // Preenche os campos do formulário automaticamente
    document.getElementById('rua').value    = data.logradouro;
    document.getElementById('bairro').value = data.bairro;
    document.getElementById('cidade').value = data.localidade;
    document.getElementById('estado').value = data.uf;
    document.getElementById('numero').focus(); // Cursor vai pro número

  } catch (error) {
    console.error('Erro ao buscar CEP:', error);
  }
}

// No HTML: <input id='cep' onblur='buscarCep(this.value)'>
```

9. Como Gerar o QR Code do Pedido

Usaremos a biblioteca qrcode.js no frontend — gratuita e simples de usar via CDN.

No HTML da página de confirmação do pedido:

```
<!-- Importar a biblioteca via CDN -->
<script
src="https://cdn.jsdelivr.net/npm/qrcode/build/qrcode.min.js"></script>

<!-- Div onde o QR Code vai aparecer -->
<div id="qrcode"></div>
```

No JavaScript:

```
function gerarQRCode(linkWhatsApp) {
  const container = document.getElementById('qrcode');
  container.innerHTML = '';  // Limpa QR Code anterior se houver

  QRCode.toCanvas(container, linkWhatsApp, {
    width: 250,
    color: {
      dark:  '#000000',  // Cor dos quadradinhos
      light: '#ffffff'   // Fundo branco
    }
  });
}
```

> Atenção: o QR Code é gerado 100% no frontend — não precisa de backend para isso. Só precisa montar o link do WhatsApp corretamente (ver seção 10).

10. Mensagem Automática do WhatsApp

O WhatsApp aceita links no formato wa.me que já abrem uma conversa com mensagem pré-preenchida. Você só precisa montar a URL corretamente.

Formato da URL:

```
https://wa.me/55NUMERO?text=MENSAGEM_CODIFICADA
```

Função completa para montar o link (carrinho.js):

```
function gerarLinkWhatsApp(pedido) {
  const telefone = '5571999990000'; // Número da lanchonete (sem +)

  // Monta a lista de itens do pedido
  let itens = pedido.itens.map(item =>
    `  - ${item.nome} x${item.quantidade} = R$ ${(item.preco *
item.quantidade).toFixed(2)}`
  ).join('\n');

  // Monta a mensagem completa
  const mensagem = `
*🍔 NOVO PEDIDO — Lanchonete Express*

*Itens:*
${itens}

*Total: R$ ${pedido.total.toFixed(2)}*

*Endereço de entrega:*
${pedido.endereco}

*Observações:* ${pedido.observacao || 'Nenhuma'}
  `.trim();

  // encodeURIComponent transforma espaços e acentos para a URL
  return `https://wa.me/${telefone}?text=${encodeURIComponent(mensagem)}`;
}

// Uso: gerarQRCode( gerarLinkWhatsApp(pedido) );
```

11. Como Implementar o Carrinho de Compras

O carrinho será salvo no localStorage do navegador — não precisa de backend. Cada item terá id, nome, preço e quantidade.

11.1 Estrutura dos dados no localStorage

```
// O carrinho é um array de objetos:
[
  { id: 1, nome: 'X-Burguer',   preco: 18.90, quantidade: 2 },
  { id: 3, nome: 'Coca-Cola 2L', preco: 12.00, quantidade: 1 }
]
```

11.2 Funções do carrinho (carrinho.js)

```
// Carregar carrinho do localStorage
function getCarrinho() {
  return JSON.parse(localStorage.getItem('carrinho')) || [];
}

// Salvar carrinho no localStorage
function salvarCarrinho(carrinho) {
  localStorage.setItem('carrinho', JSON.stringify(carrinho));
  atualizarContadorCarrinho(); // Atualiza o ícone do carrinho
}

// Adicionar produto ao carrinho
function adicionarAoCarrinho(produto) {
  let carrinho = getCarrinho();
  const index = carrinho.findIndex(item => item.id === produto.id);

  if (index !== -1) {
    carrinho[index].quantidade++;   // Já existe: aumenta quantidade
  } else {
    carrinho.push({ ...produto, quantidade: 1 }); // Novo item
  }
  salvarCarrinho(carrinho);
}

// Calcular total do carrinho
function calcularTotal() {
  return getCarrinho()
    .reduce((total, item) => total + item.preco * item.quantidade, 0)
    .toFixed(2);
}

// Esvaziar carrinho após pedido
function limparCarrinho() {
  localStorage.removeItem('carrinho');
}
```

O localStorage persiste mesmo fechando o navegador. Isso significa que o cliente pode fechar a aba e voltar depois com o carrinho ainda preenchido, ótima experiência!

12. Como Implementar o Painel Administrativo

O painel admin será uma página separada (admin/index.html) protegida por autenticação via token JWT.

12.1 Proteção da Rota no Frontend

```
// No início de qualquer página admin:
const tokenAdmin = localStorage.getItem('adminToken');
if (!tokenAdmin) {
  window.location.href = '/admin/login.html'; // Redireciona se não logado
}
```

12.2 Proteção das Rotas no Backend (Middleware)

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

function verificarAdmin(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ erro: 'Token não fornecido' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    if (decoded.tipo !== 'admin') return res.status(403).json({ erro:
'Acesso negado' });
    req.admin = decoded;
    next();
  } catch {
    return res.status(401).json({ erro: 'Token inválido' });
  }
}

module.exports = { verificarAdmin };
```

12.3 Funcionalidades do Painel

- Dashboard: Cards com total de pedidos do dia, faturamento e produtos cadastrados
- Gerenciar Produtos: Tabela com todos os produtos + botões Editar e Remover
- Adicionar Produto: Formulário com campos nome, preço, categoria, imagem e descrição
- Ver Pedidos: Tabela com cliente, itens, valor, status e botão para mudar status
- Promoções: Campo para definir preço promocional em qualquer produto existente

Dependências do Backend

```
npm install express cors dotenv mysql2 bcryptjs jsonwebtoken
```

Pacote	Para que serve
express	Framework para criar o servidor e as rotas
cors	Permite que o frontend faça chamadas para a API
dotenv	Carrega variáveis do arquivo .env
mysql2	Conectar e fazer queries no banco MySQL
bcryptjs	Criptografar senhas antes de salvar no banco
jsonwebtoken	Gerar e verificar tokens de autenticação (JWT)